

SYNOPSIS ON:
CONTAINER IMAGE THREAT SCANNER WITH LAYER-AWARE
FORENSICS.

BACHELOR OF COMPUTER APPLICATIONS
OF

University of mysore



Harshitha Shree D U01CO23S0201

Spandana R S U01CO23S0202

Yogesh S U01CO23S0203

Pavan N U01CO23S0204

Under the guidance of :

ANNAPOORNA M S

Assistance Professor, Dept of BCA Mysore



DEPARTMENT OF BACHELOR OF COMPUTRES APPLICATIONS

BGS FIRST GRADE COLLEGE

MYSORE-23

2025-2026

SYNOPSIS

TITLE OF THE PROJECT

Container Image Threat Scanner with Layer-Aware Forensics

Aim of the Project

The aim of this project is to design and develop an Image / Container Threat Scanner that systematically analyzes container images to identify security threats such as vulnerabilities, malicious packages, insecure configurations, and supply-chain risks. The project focuses on scanning container images at a granular level to provide clear visibility into security issues introduced during image creation, enabling early detection and effective remediation. By improving transparency, accuracy, and security awareness in containerized environments, the system aims to support secure application deployment and strengthen overall DevSecOps practices.

1. INTRODUCTION

With the rapid evolution of cloud computing and microservices architecture, containerization technologies such as Docker have become a fundamental component of modern software development and deployment. Containers provide lightweight, portable, and scalable environments that enable applications to run consistently across different platforms. As a result, organizations increasingly rely on container images to build, deploy, and manage applications efficiently.

However, the widespread adoption of containers has also introduced new security challenges. Container images are constructed in multiple layers, each adding software packages, libraries, configuration files, or application code. Vulnerabilities, misconfigurations, malicious dependencies, or outdated libraries can be introduced at any layer during the image build process. These security risks can propagate into production environments, leading to data breaches, system compromise, and supply-chain attacks.

Most existing container security solutions analyze container images as a single entity and provide aggregated vulnerability reports. Such approaches fail to identify which specific layer introduced a particular vulnerability, making root-cause analysis difficult and slowing down remediation efforts.

This project proposes the design and development of a **Container Image Threat Scanner with Layer-Aware Forensics**, which performs detailed layer-by-layer analysis of Docker container

images. The system provides precise forensic insights by tracing vulnerabilities and security risks back to their originating layers, enabling faster remediation and improved DevSecOps workflows.

2. PROBLEM STATEMENT

Despite the availability of container vulnerability scanning tools, several limitations exist in current solutions. Existing container security tools often:

- Lack fine-grained layer-level vulnerability analysis
- Provide limited forensic and root-cause insights
- Fail to trace vulnerable dependencies to their exact origin
- Offer insufficient remediation guidance for developers

These shortcomings result in inefficient vulnerability management and increased security risks in containerized environments.

3. OBJECTIVES

The primary objectives of this project are:

- To analyze Docker container images at an individual layer level
- To detect vulnerabilities, insecure packages, and malicious artifacts
- To trace the origin of vulnerabilities to specific image layers
- To generate detailed forensic reports with remediation guidance
- To support secure DevSecOps and CI/CD practices

4. SCOPE OF THE PROJECT

Included Scope

- Static analysis of Docker container images
- Layer-by-layer inspection of filesystem changes
- Detection of vulnerable dependencies using CVE databases
- Generation of Software Bill of Materials (SBOM)

Excluded Scope

- Runtime container behavior monitoring
- Kernel-level container escape detection

- Live intrusion detection
- Hardware-based security analysis

5. PROPOSED SYSTEM ARCHITECTURE

The proposed system consists of the following modules:

1. Image Extraction Module

- Pulls Docker container images from local or remote registries
- Extracts and unpacks filesystem layers

2. Layer Analysis Engine

- Identifies files added or modified in each layer
- Maps installed packages and binaries to corresponding layers

3. Vulnerability Detection Module

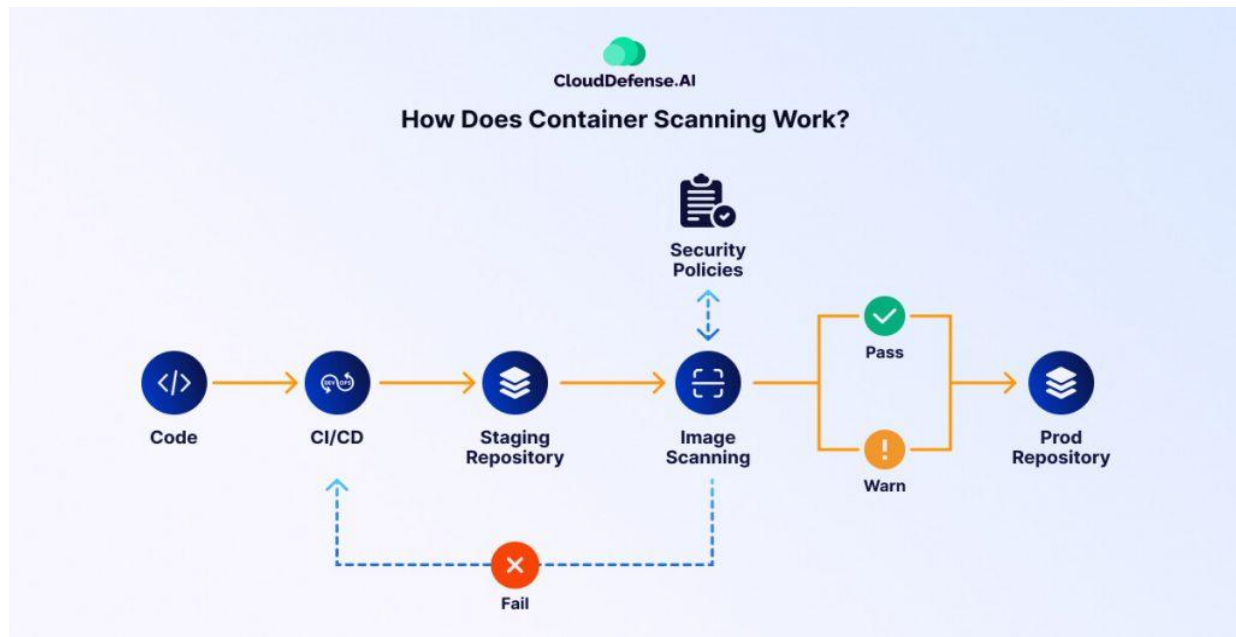
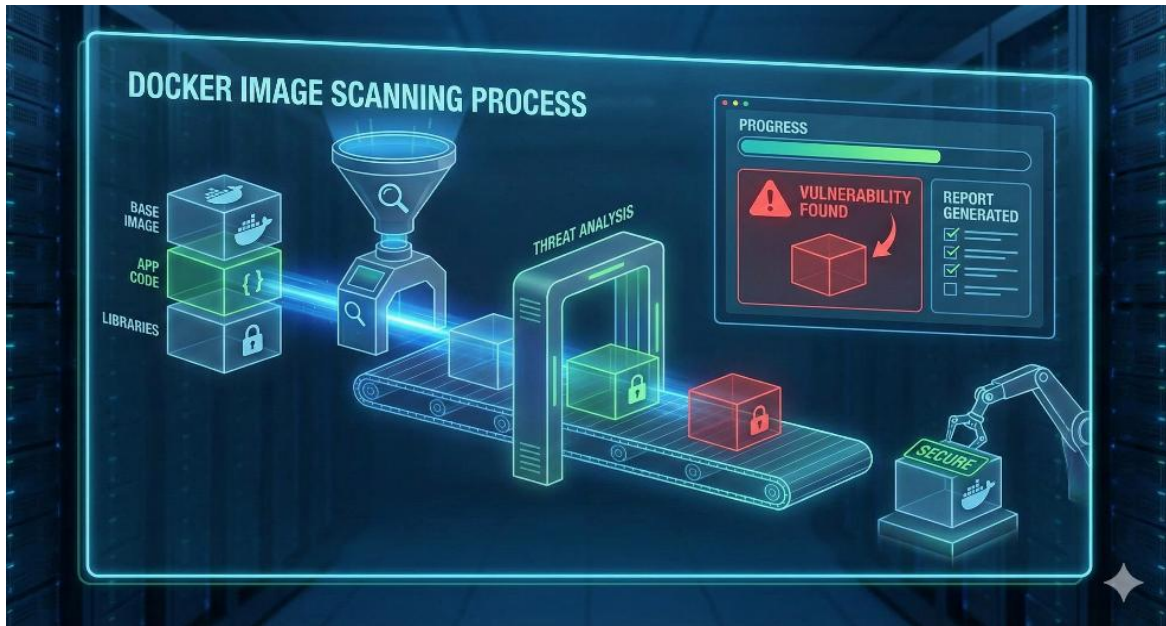
- Matches dependencies against known CVE databases
- Identifies outdated and insecure libraries

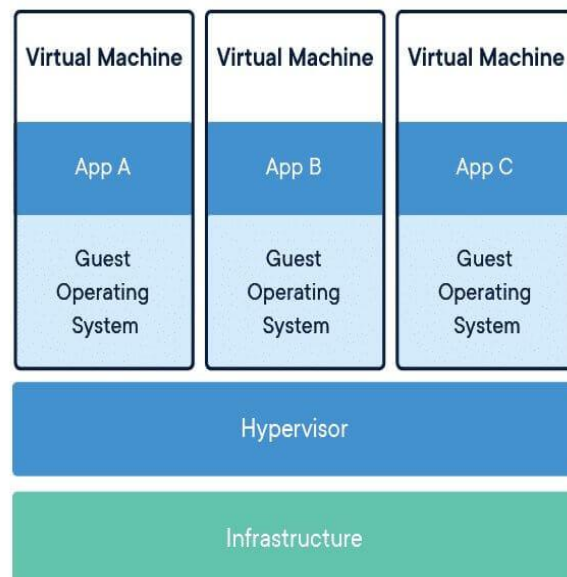
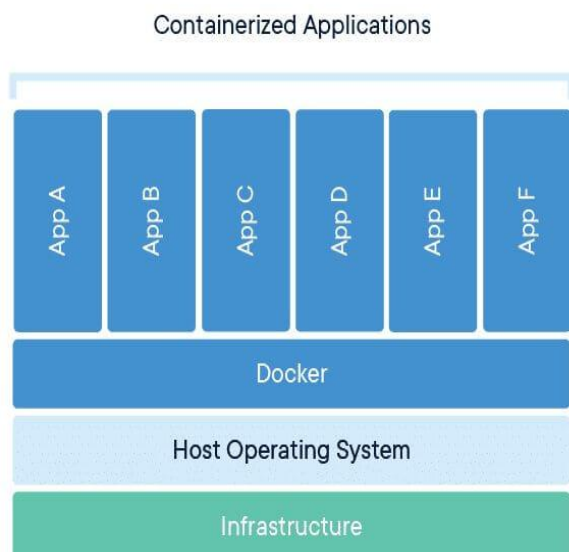
4. Threat Intelligence Engine

- Detects suspicious binaries, scripts, and configurations
- Flags high-risk artifacts and misconfigurations

5. Forensic Report Generator

- Produces structured security and forensic reports
- Provides actionable remediation recommendations





DOCKER IMAGE LAYERS

www.learnitguide.net

1. Started Image Layer 3 as a container and accessible by users

1. Started Image Layer v1 as a container.
2. Installed and Configured https web server.
3. Committed new layer v2

1. Started Base Image (**docker.io/centos**) as a container.
2. Package Updated on Base Image using "yum update".
3. Committed new layer v1

Pulled CentOS image from Docker Hub using docker pull command. Repo: **docker.io/centos**



6. METHODOLOGY

The methodology of the **Container Image Threat Scanner with Layer-Aware Forensics** follows a systematic and modular approach to ensure accurate identification, tracing, and reporting of security risks within Docker container images. The entire process is divided into well-defined stages, each responsible for a specific security analysis task.

Step 1: Container Image Acquisition

The process begins by acquiring the target Docker container image from a local repository or a remote container registry such as Docker Hub. The image can be specified by its name and version tag. Once identified, the Docker engine pulls the image and stores it locally for analysis. This ensures that the scanner operates on the exact image intended for deployment.

Step 2: Image Decomposition into Layers

Docker images are composed of multiple read-only layers, each created by an instruction in the Docker file. In this step, the pulled image is decomposed into its individual filesystem layers. Each layer represents a snapshot of filesystem changes such as newly added files, installed packages, or configuration updates. Extracting layers separately allows the system to perform granular inspection rather than treating the image as a single unit.

Step 3: Layer-wise Filesystem Inspection

Each extracted layer is analyzed independently to identify files and directories added or modified at that stage. The system records file metadata such as file type, size, permissions, and ownership. By comparing layers, the scanner identifies the exact layer in which specific files or binaries were introduced. This layer-wise inspection forms the foundation for forensic traceability.

Step 4: Package and Dependency Enumeration

In this stage, the system detects installed software packages and libraries within each layer using operating system-specific package managers such as apt, yum, or apk. The scanner extracts package names, versions, and dependency relationships. This information is organized on a per-layer basis, enabling the generation of a Software Bill of Materials (SBOM) that lists all dependencies introduced at each layer of the container image.

Step 5: Vulnerability Identification Using CVE Databases

The enumerated packages and libraries are compared against publicly available vulnerability databases such as the Common Vulnerabilities and Exposures (CVE) repository. The system matches software versions with known vulnerabilities and records the severity, description, and impact of each identified issue. Each vulnerability is explicitly mapped to the layer that introduced the affected package, enabling precise root-cause analysis.

Step 6: Threat Intelligence and Misconfiguration Analysis

Beyond known vulnerabilities, the system performs additional security checks to identify potential threats and insecure configurations. This includes scanning for suspicious binaries, unsafe scripts, hardcoded credentials, exposed configuration files, and insecure permissions. Rule-based detection mechanisms are used to flag high-risk artifacts that may indicate malicious intent or poor security practices.

Step 7: Risk Correlation and Forensic Mapping

All identified vulnerabilities and threats are correlated with their respective layers and associated files or packages. This correlation process enables forensic mapping, which clearly traces each security issue back to its origin within the container image build process. The system maintains a structured mapping of risks, layers, and components to ensure transparency and auditability.

Step 8: Forensic Report Generation

In the final stage, the system generates comprehensive forensic reports in structured formats such as JSON or YAML. The reports include details of detected vulnerabilities, affected packages, originating layers, severity levels, and recommended remediation steps. These reports provide actionable insights that help developers and security teams rebuild or fix only the affected layers, thereby improving remediation speed and security efficiency.

Step 9: Optional Visualization and Integration

Optionally, the generated reports can be visualized using a lightweight web-based dashboard developed with Flask or Streamlit. This interface allows users to view layer-wise risk summaries, vulnerability distributions, and remediation suggestions. The system can also be integrated into CI/CD pipelines to perform automated container image security checks during the build and deployment phases.

7. TOOLS AND TECHNOLOGIES USED

- **Docker** – Container image handling and extraction
- **Python** – Core scanning and analysis logic
- **SBOM Tools** – Dependency enumeration
- **CVE Databases** – Vulnerability identification
- **Linux Utilities** – Filesystem inspection
- **JSON / YAML** – Report formatting
- **Flask** – Web-based visualization dashboard

8. SYSTEM REQUIREMENTS

1. Hardware Requirements

The following hardware configuration is required for the effective development and execution of the proposed system:

- **Processor:** Intel Core i5 (8th Generation) or above

- **Processor Speed:** 2.4 GHz or higher
- **Main Memory (RAM):** Minimum 8 GB (16 GB recommended for large images)
- **Hard Disk:** Minimum 50 GB free storage
- **System Type:** 64-bit Architecture
- **Network:** Active Internet connection (for pulling images and CVE updates)

2. Software Requirements

The following software components are required for the implementation of the project:

- **Operating System:**
 - Linux (Ubuntu 20.04 or above) – *Preferred*
 - Windows 10/11 (with WSL2 support)
- **Container Platform:**
 - Docker Engine (version 20.x or above)
- **Programming Language:**
 - Python 3.9 or above
- **Libraries and Frameworks:**
 - Docker SDK for Python
 - Requests (for CVE data retrieval)
 - JSON / YAML libraries
 - OS and filesystem utilities
- **SBOM Tools:**
 - Syft or equivalent dependency enumeration tools
- **Vulnerability Databases:**
 - National Vulnerability Database (NVD) / CVE feeds
- **Web Framework (Optional):**
 - Flask or Streamlit (for dashboard visualization)
- **Development Tools:**
 - Visual Studio Code / PyCharm
 - Git (Version Control)

9. EXPECTED OUTCOMES

- A functional container image threat scanning system
- Layer-wise visibility of vulnerabilities and security risks
- Detailed forensic reports with actionable insights
- Faster and more accurate vulnerability remediation

10. ADVANTAGES

- Fine-grained layer-level threat visibility
- Faster root-cause identification of vulnerabilities
- Improved remediation accuracy
- Enhanced container supply-chain security

11. CONCLUSION

The proposed **Container Image Threat Scanner with Layer-Aware Forensics** addresses critical security gaps in modern containerized environments. By enabling precise layer-level vulnerability tracking and forensic analysis, the system significantly enhances container security, improves DevSecOps workflows, and mitigates software supply-chain risks. This project contributes toward building more secure, transparent, and resilient container-based application deployments.

12. Reference

1. P. P. Kaur, R. Kumar, “**Security Challenges in Docker Containers and Their Mitigation,**” *International Journal of Computer Applications*, vol. 176, no. 15, 2020.
2. NIST, “**Application Container Security Guide,**” *NIST Special Publication 800-190*, National Institute of Standards and Technology, USA, 2017.
3. S. Pahl, “**Containerization and the PaaS Cloud,**” *IEEE Cloud Computing*, vol. 2, no. 3, pp. 24–31, 2015.
4. Anchore Inc., “**Container Image Security Best Practices,**” *White Paper*, 2021.
5. Aqua Security, “**Container Image Vulnerability Scanning and Analysis,**” *Technical Documentation*, 2022.
6. OWASP Foundation, “**OWASP Top 10 for Docker and Container Security Risks,**” OWASP Project, 2023.
7. J. Turnbull, “**The Docker Book: Containerization Is the New Virtualization,**” James Turnbull Publishing, 2019.
8. Snyk Ltd., “**State of Open Source Security and Container Vulnerabilities,**” *Security Report*, 2022.
9. NIST, “**Software Bill of Materials (SBOM) Framework for Supply Chain Risk Management,**” NIST Cybersecurity White Paper, 2021.
10. Red Hat, “**Understanding Container Image Layers and Security Risks,**” *Red Hat Developer Blog*, 2022.
11. Clair Project Team, “**Clair: Static Analysis of Vulnerabilities in Container Images,**” *CoreOS Documentation*, 2020.
12. Trivy Authors, “**Trivy: A Comprehensive Vulnerability Scanner for Containers and SBOMs,**” *Open Source Documentation*, 2023